VHDL Assignment 2 실습 보고서

김태은 2023090164

과제 1. 4입력 AND / OR 논리 연산

1. 실습 목적

네 개의 입력에 대한 AND와 OR 연산을 VHDL로 작성하고, Logisim-evolution 시뮬레이션으로 출력 결과를 확인한다.

2. 이론

AND 연산은 모든 입력이 1일 때만 1을 출력하며, OR 연산은 입력 중 하나라도 1이면 1을 출력한다. 두 연산 모두 현재 입력값만으로 출력이 결정되는 조합논리회로이다.

3. VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
entity Assignment2_1 is
    port (
        a, b, c, d : in std_logic;
        and_out, or_out : out std_logic
    );
end entity Assignment2_1;

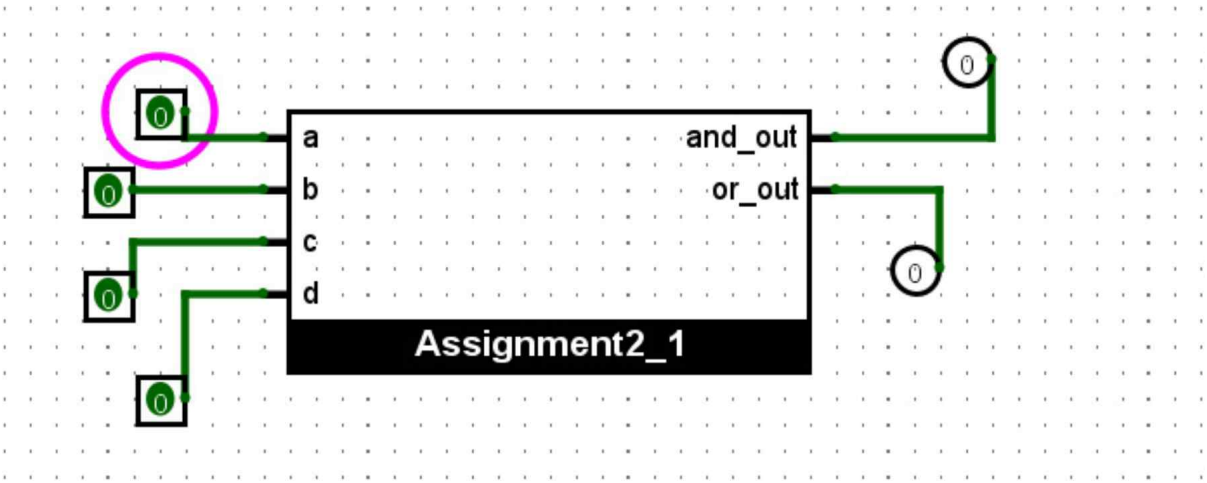
--architecture
architecture bhv of Assignment2_1 is
begin
    and_out <= a and b and c and d;
    or_out  <= a or b or c or d;
end architecture bhv;
```

4. 회로 구성

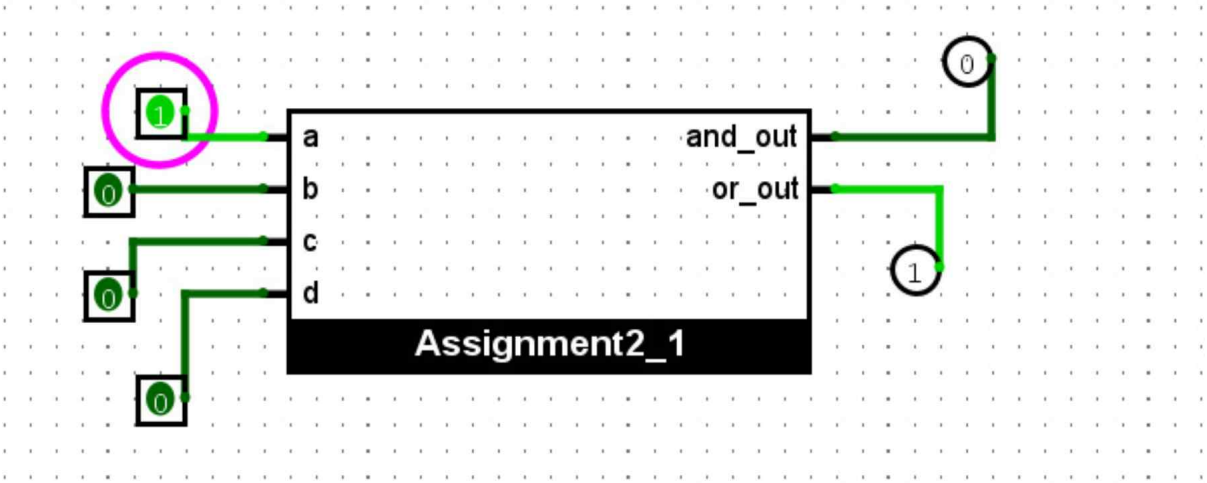
입력 a, b, c, d를 VHDL 블록에 연결하고 출력 and_out, or_out을 각각 출력 핀으로 확인하였다. 출력 이름은 과제 원문의 out1, out2와 다르지만 연산은 동일하다.

5. 실습 결과

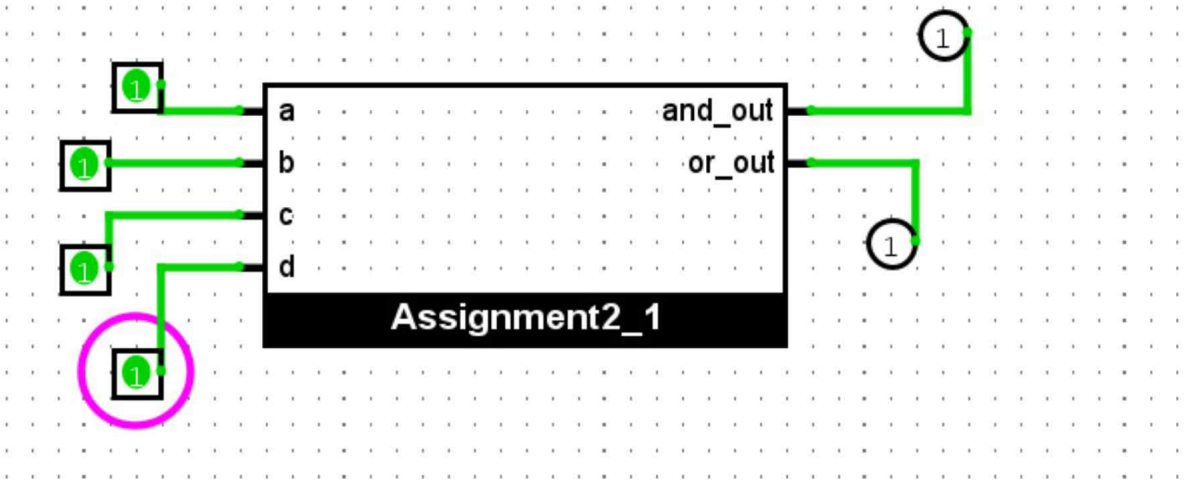
0000에서는 AND와 OR 모두 0, 1000에서는 AND 0·OR 1, 1111에서는 AND와 OR 모두 1이 출력되었다.



a b c d = 0000 → AND = 0, OR = 0



a b c d = 1000 → AND = 0, OR = 1



a b c d = 1111 → AND = 1, OR = 1

과제 2. 8-to-1 Multiplexer

1. 실습 목적

8개의 입력 a~h 중 3비트 선택 신호 ch가 지정한 하나의 입력을 출력 data로 전달하는 8:1 MUX를 구현한다.

2. 이론

멀티플렉서는 여러 입력 중 선택 신호가 가리키는 한 개의 입력을 출력으로 전달하는 조합논리회로이다. 8:1 MUX는 3비트 선택 신호로 8개 입력 중 하나를 선택하며, 선택되지 않은 입력은 출력에 영향을 주지 않는다.

3. VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

entity Assignment2_2 is
    port (
        a, b, c, d, e, f, g, h : in std_logic;
        ch : in std_logic_vector(2 downto 0);
        data : out std_logic
    );
end entity Assignment2_2;

architecture bhv of Assignment2_2 is
begin
    process(a,b,c,d,e,f,g,h,ch)
    begin
        case ch is
            when "000" => data <= a;
            when "001" => data <= b;
            when "010" => data <= c;
            when "011" => data <= d;
            when "100" => data <= e;
            when "101" => data <= f;
            when "110" => data <= g;
            when "111" => data <= h;
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

4. 회로 구성

8개의 1비트 입력과 3비트 선택 신호 ch를 VHDL 블록에 연결하고 data를 출력 핀으로 확인하였다.

5. 실습 결과

선택 신호를 고정한 상태에서 해당 입력을 0에서 1로 변경하면 출력도 동일하게 변경되었다.

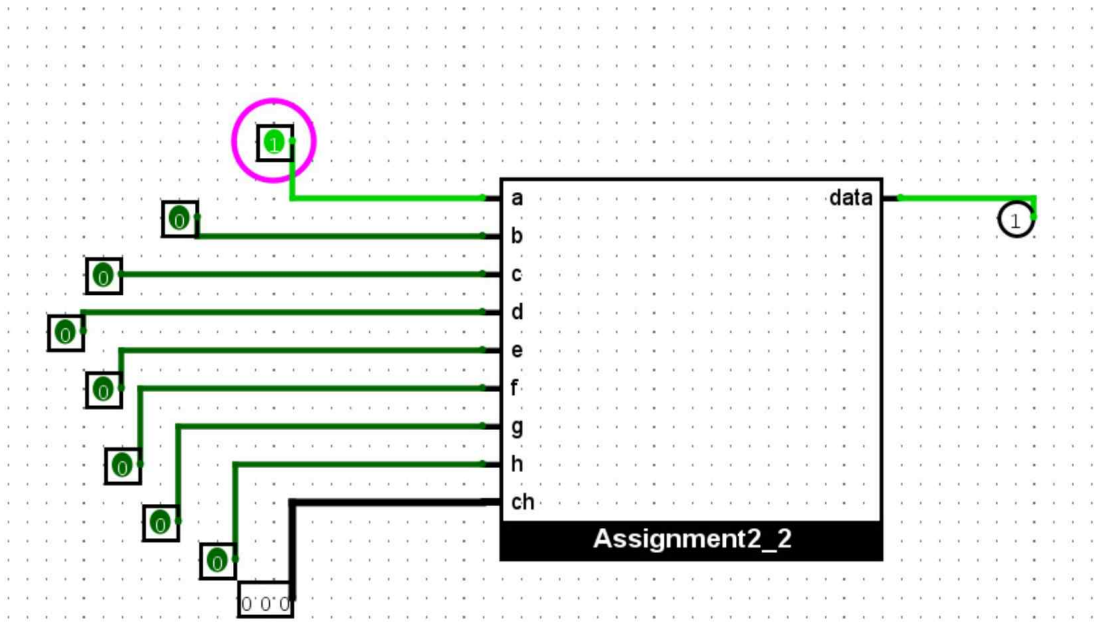
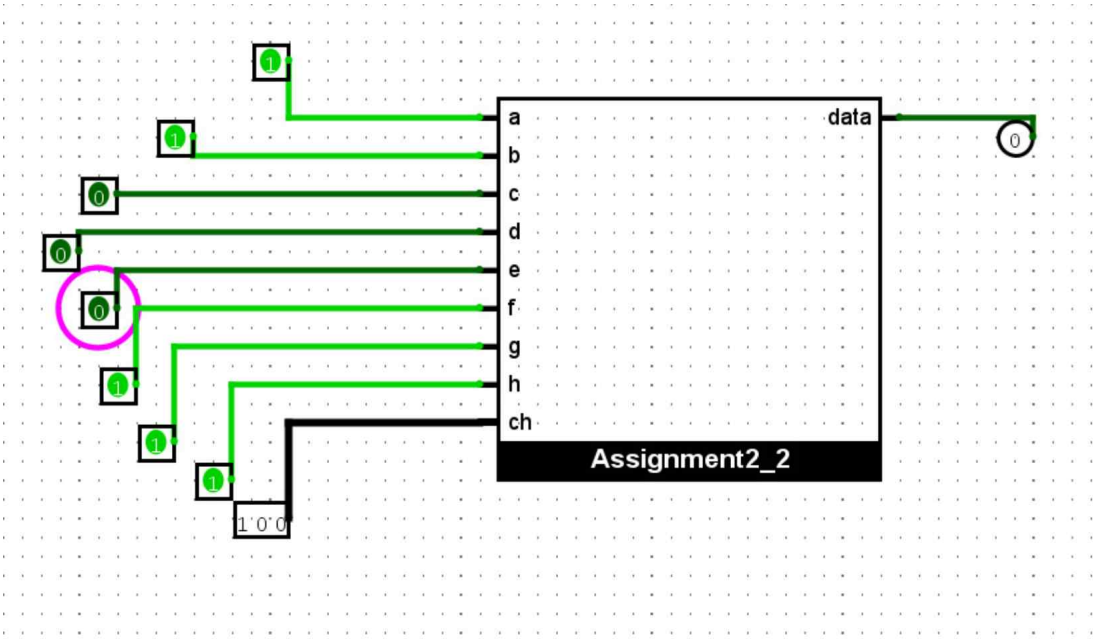
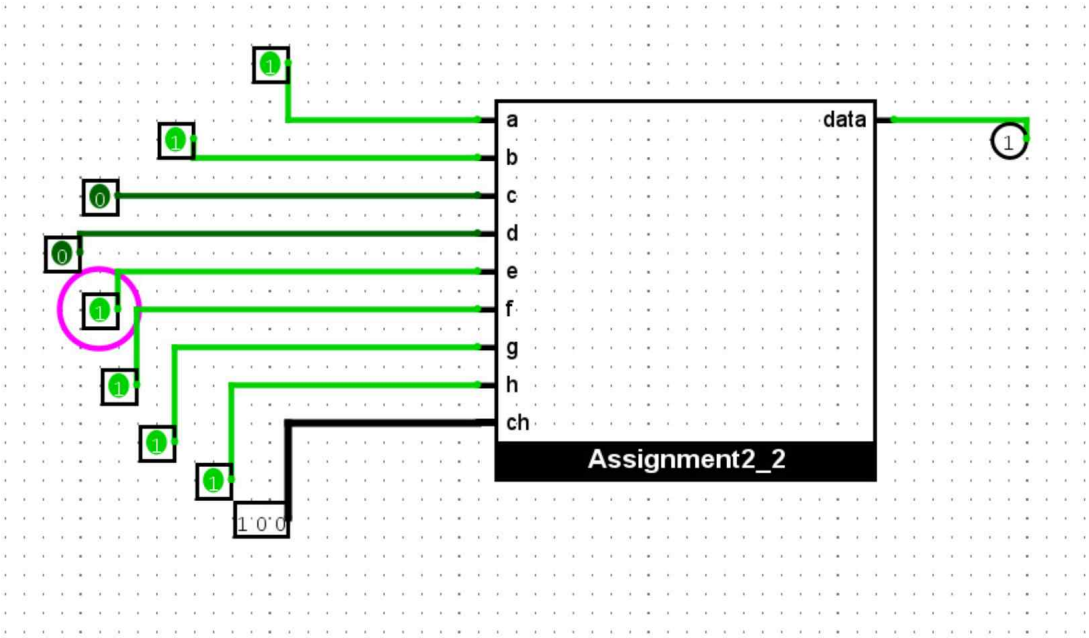


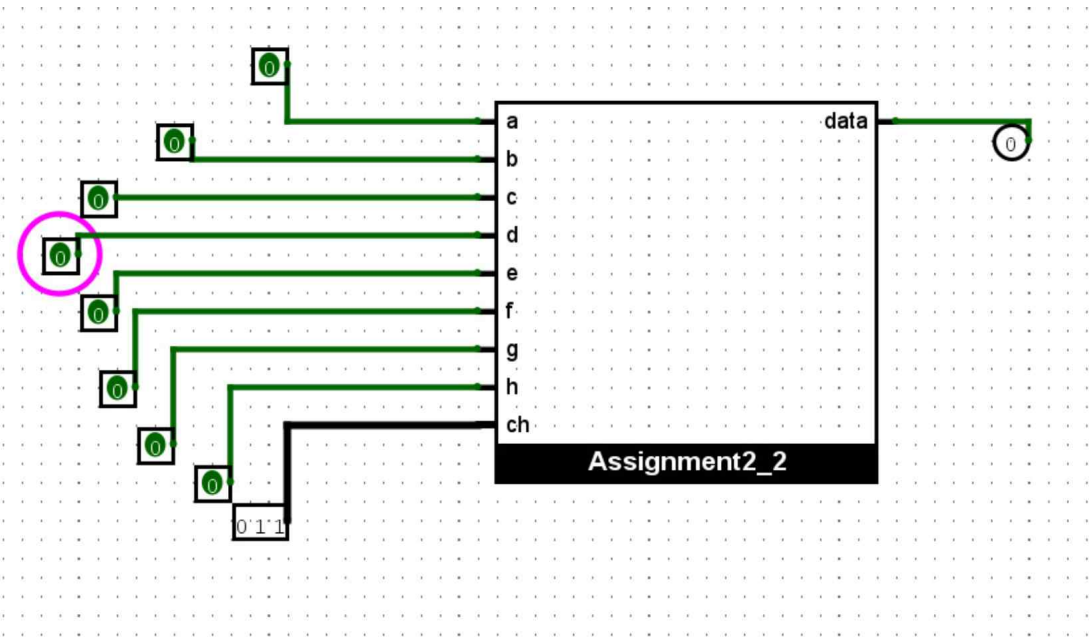
그림 2-1 ch = 000, a = 1 → data = 1



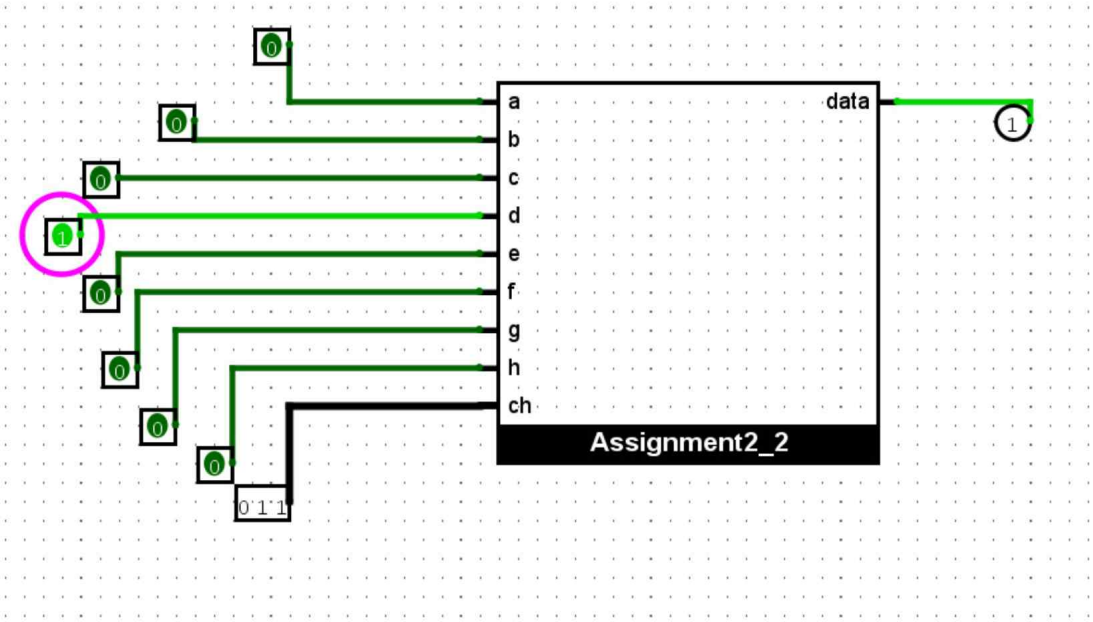
ch = 100, e = 0 → data = 0



ch = 100, e = 1 → data = 1



ch = 011, d = 0 → data = 0



ch = 011, d = 1 → data = 1

과제 3. 7-Segment 문자 표시

1. 실습 목적

7-Segment로 과제에서 요구한 a, b, c, d, f, g의 여섯 문자를 출력하고 입력 dat에 따른 문자 전환을 확인한다.

2. 이론

7-Segment 표시기는 일곱 개의 발광 구간을 조합하여 숫자나 일부 알파벳을 표시하는 장치이다. VHDL에서 입력값에 맞는 점등 패턴을 출력하면 해당 문자가 표시되며, en 신호로 표시 여부를 제어할 수 있다.

3. VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

entity Assignment2_3 is
    port (
        en : in std_logic;
        dat : in std_logic_vector(2 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end entity Assignment2_3;

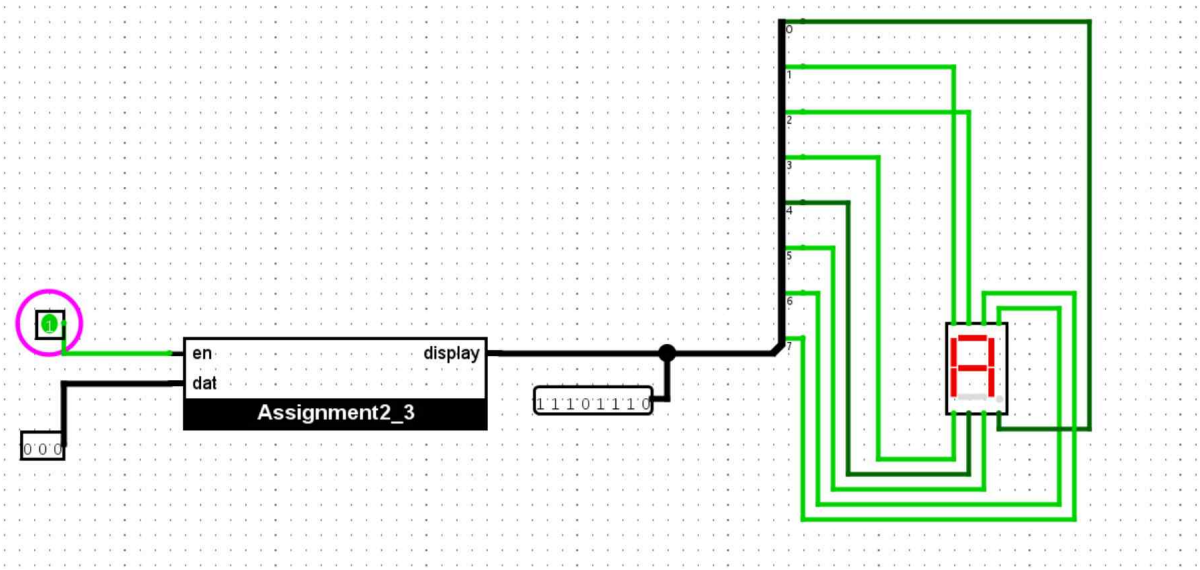
architecture bhv of Assignment2_3 is
begin
    process(en, dat)
    begin
        if en = '0' then
            display <= "00000000";
        else
            case dat is
                when "000" => display <= "11101110"; -- A
                when "001" => display <= "00111110"; -- b
                when "010" => display <= "10011100"; -- C
                when "011" => display <= "01111010"; -- d
                when "100" => display <= "10001110"; -- F
                when "101" => display <= "10111110"; -- G
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

4. 회로 구성

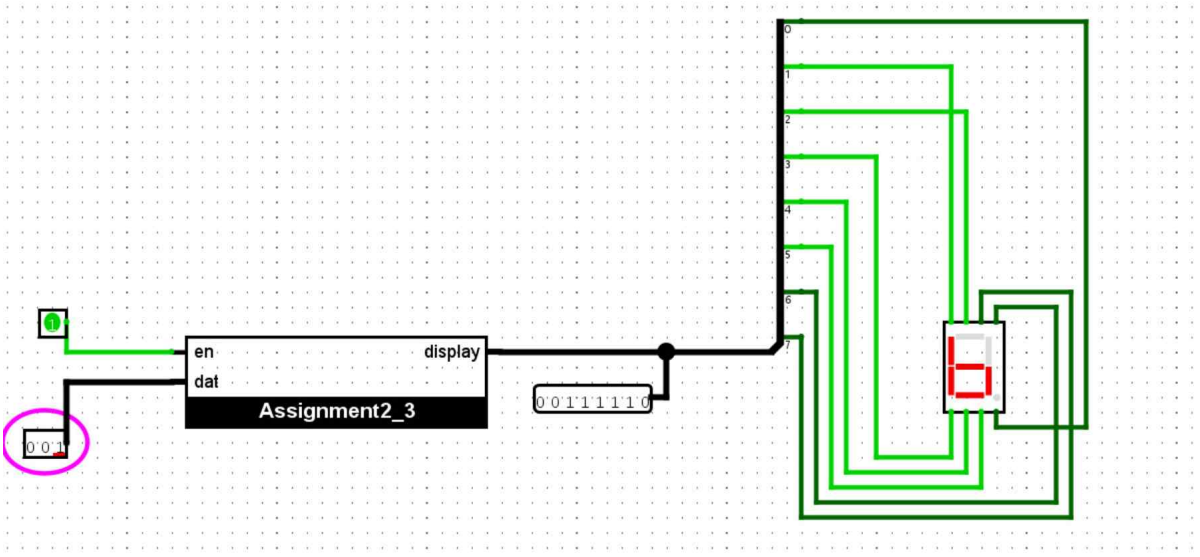
입력 en(1비트), dat(3비트)을 VHDL 블록에 연결하고, 8비트 display 출력을 각 세그먼트에 연결하였다. 7-Segment의 표기 특성에 따라 A, b, C, d, F, G 형태로 표시한다.

5. 실습 결과

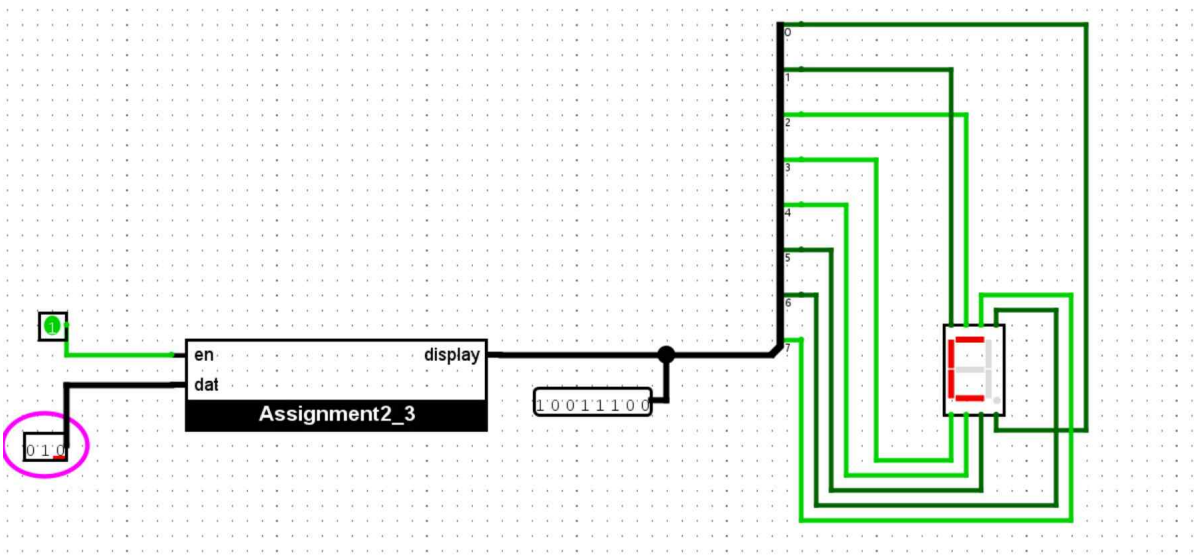
en=1에서 dat=000~101에 따라 6개 문자가 표시되었다. dat=111은 미지정 값이어서 꺼졌고, en=0에서도 소등되었다.



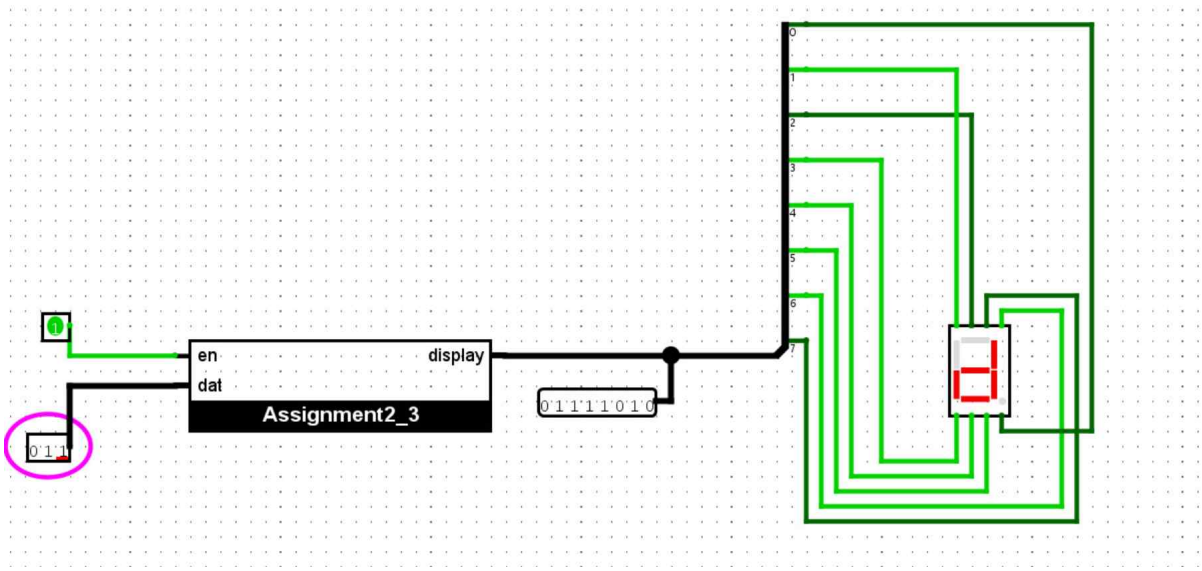
en = 1, dat = 000 → A



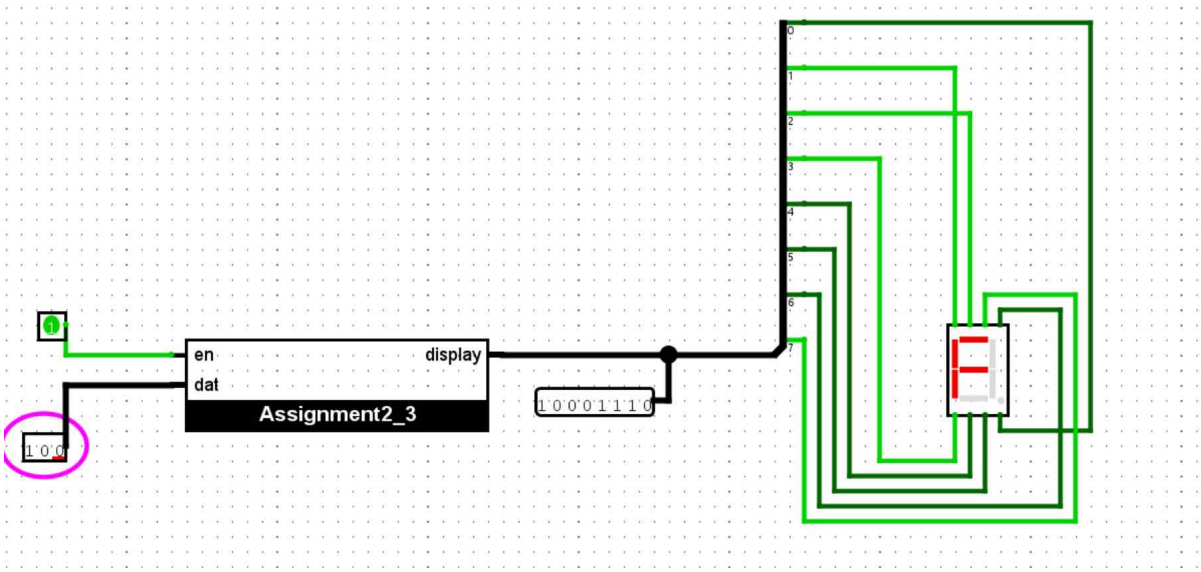
en = 1, dat = 001 → b



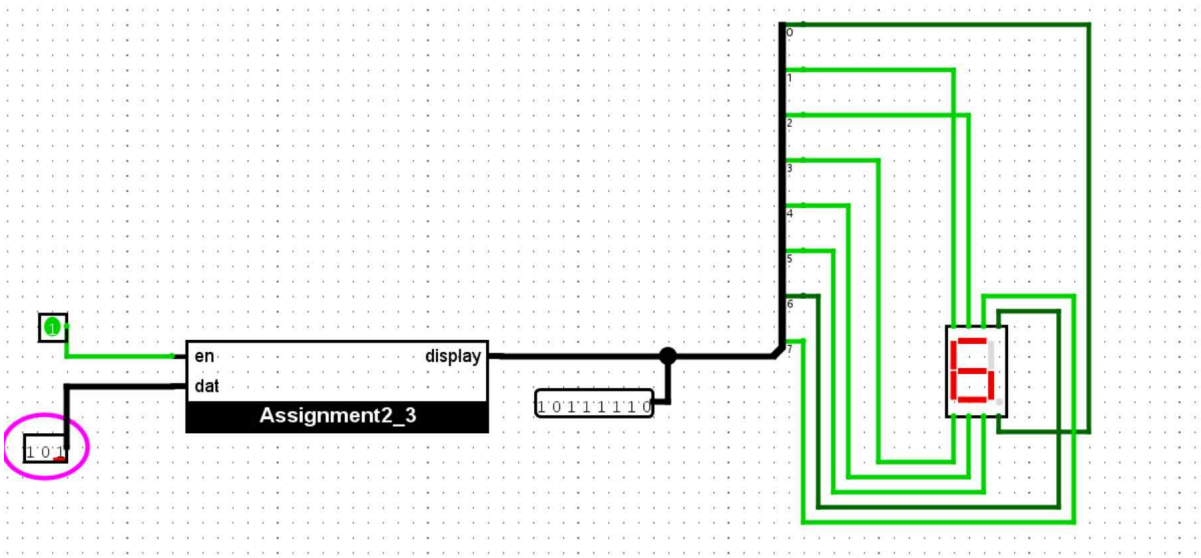
en = 1, dat = 010 → C



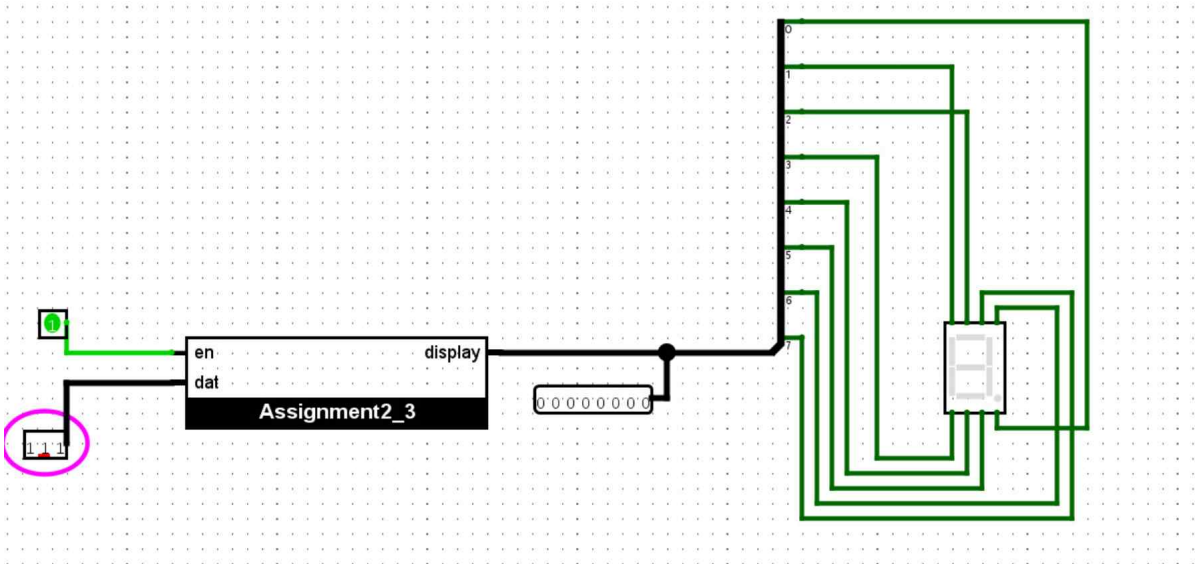
en = 1, dat = 011 → d



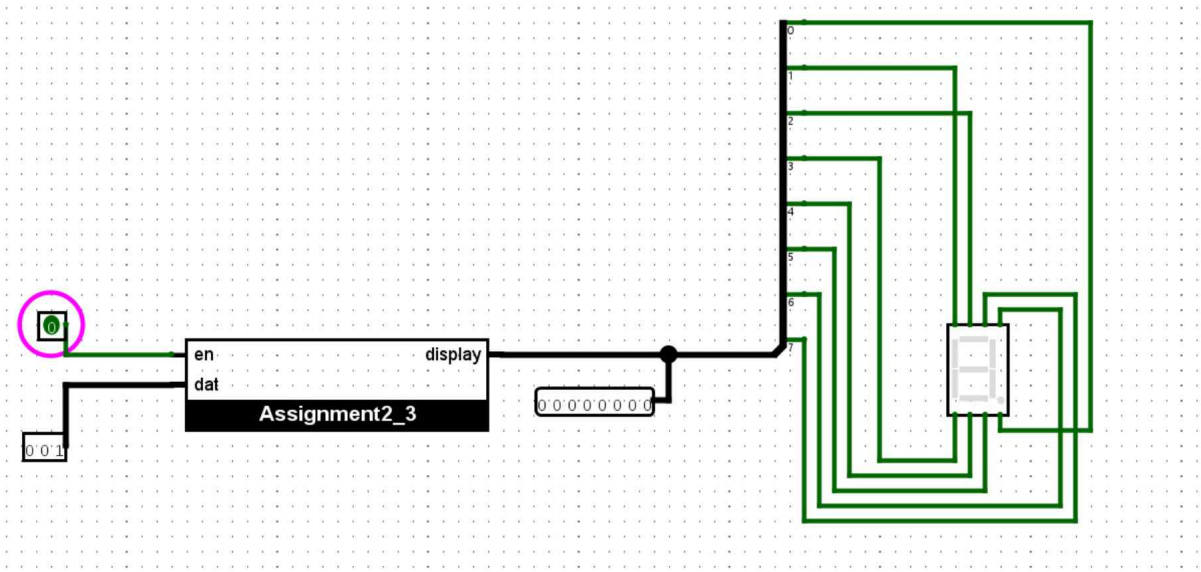
en = 1, dat = 100 → F



en = 1, dat = 101 → G



en = 1, dat = 111 → 소등



en = 0, dat = 001 → 소등

과제 4. 25명 다수결 투표 회로

1. 실습 목적

25명의 찬성 여부를 25비트 입력으로 받아 찬성표 수를 계산하고, 과반수인 13표 이상을 통과로 판정하는 회로를 구현한다.

2. 이론

다수결 회로는 각 투표 입력을 0 또는 1로 처리하고, 1인 입력의 개수를 합산해 결과를 판정한다. 총 25명일 때 과반수는 13명이므로 찬성표가 13표 이상이면 1, 그보다 적으면 0을 출력한다.

3. VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Assignment2_4 is
    port (
        vote   : in std_logic_vector(24 downto 0);
        result : out std_logic
    );
end entity Assignment2_4;

architecture bhv of Assignment2_4 is
begin
    process(vote)
        variable count : integer range 0 to 25;
    begin
        count := 0;

        for i in 0 to 24 loop
            if vote(i) = '1' then
                count := count + 1;
            end if;
        end loop;

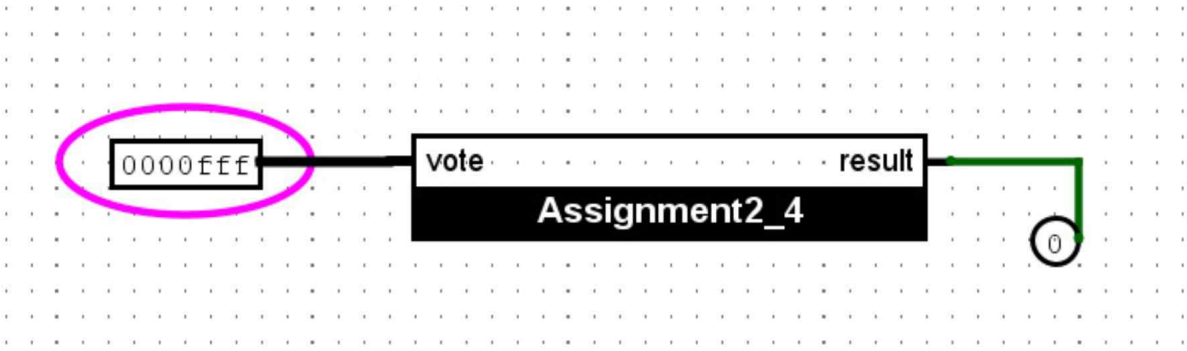
        if count >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

4. 회로 구성

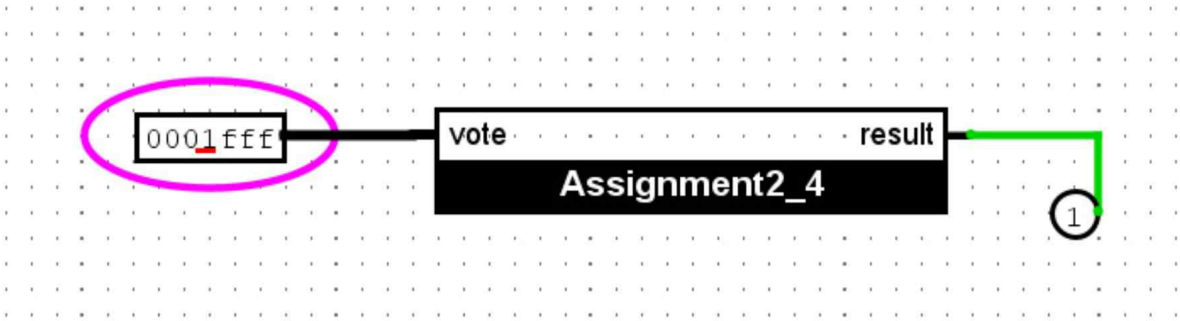
25비트 vote 입력을 16진수로 설정하고, 결과를 1비트 result 출력에 표시하였다. 찬성표가 12개 이하면 0, 13개 이상이면 1이다.

5. 실습 결과

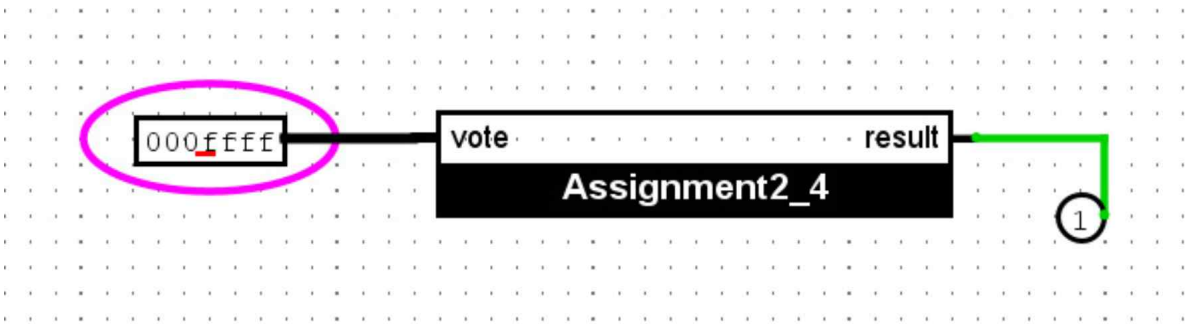
12표와 13표를 비교하여 과반수 경계가 정확히 처리되는지 확인하였다. 1표와 16표 조건에서도 결과가 판정 기준과 일치하였다.



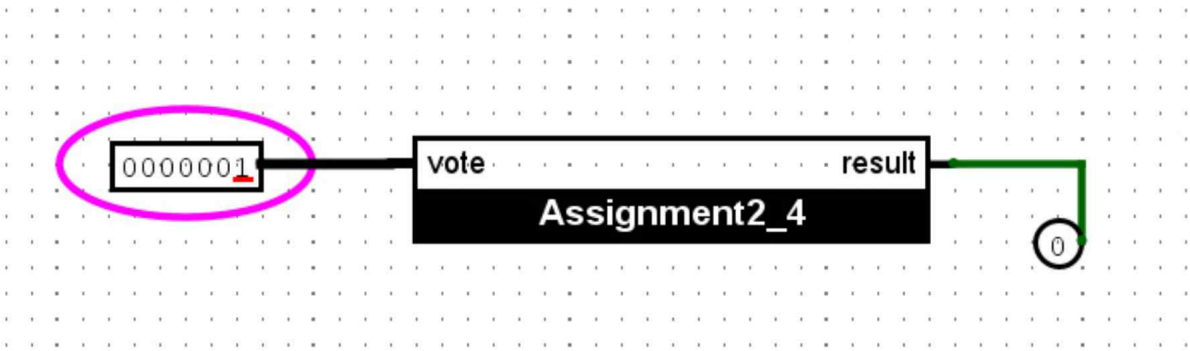
vote = 0000FFF (12표) → result = 0



vote = 0001FFF (13표) → result = 1



vote = 000FFFF (16표) → result = 1



vote = 0000001 (1표) → result = 0

과제 5. 13비트 입력의 1 개수와 7-Segment 출력

1. 실습 목적

13비트 vt에 포함된 1의 개수를 계산하여 0~9는 숫자로, 10 이상은 문자로 표시하고 en 신호에 따른 표시 활성화 동작을 확인한다.

2. 이론

입력 벡터에서 1인 비트의 개수를 세는 연산을 비트 개수 계산(popcount)이라고 하며, 계산 결과를 7-Segment 점등 패턴으로 변환할 수 있다. 13비트에서는 0~13개만 발생하므로 E와 F에 대응하는 14·15개 상태는 입력만으로 도달하지 않는다.

3. VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Assignment2_5 is
    port (
        vt        : in  std_logic_vector(12 downto 0);
        en        : in  std_logic;
        count_out : out std_logic_vector(3 downto 0);
        display   : out std_logic_vector(7 downto 0)
    );
end entity Assignment2_5;

architecture bhv of Assignment2_5 is
begin
    process(vt, en)
        variable count : integer range 0 to 15;
    begin
        count := 0;

        for i in vt'range loop
            if vt(i) = '1' then
                count := count + 1;
            end if;
        end loop;

        count_out <= std_logic_vector(
            to_unsigned(count, 4)
        );


        if en = '0' then
            display <= "00000000";
        else
            case count is
                when 0  => display <= "11111100"; -- 0
                when 1  => display <= "01100000"; -- 1
                when 2  => display <= "11011010"; -- 2
                when 3  => display <= "11110010"; -- 3
                when 4  => display <= "01100110"; -- 4
                when 5  => display <= "10110110"; -- 5
                when 6  => display <= "10111110"; -- 6
                when 7  => display <= "11100000"; -- 7
                when 8  => display <= "11111110"; -- 8
                when 9  => display <= "11110110"; -- 9

                when 10 => display <= "11101110"; -- A
                when 11 => display <= "00111110"; -- b
                when 12 => display <= "10011100"; -- C
                when 13 => display <= "01111010"; -- d
                when 14 => display <= "10011110"; -- E
                when 15 => display <= "10001110"; -- F

                when others =>
                    display <= "00000000";
            end case;
```

```
        end if;
    end process;
end architecture bhv;
```

4. 회로 구성

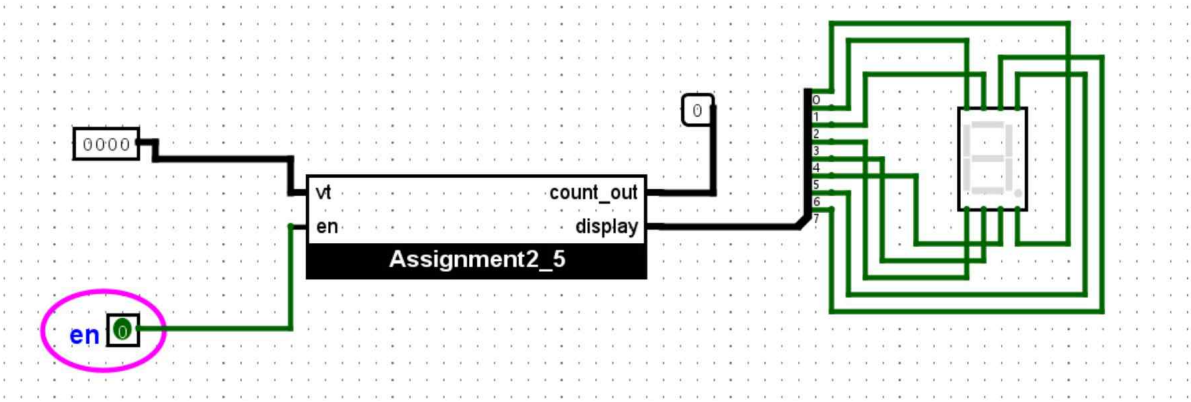
13비트 vt, 1비트 en을 입력에 연결하였다. 4비트 count_out에서 계산값을 확인하고, 8비트 display를 7-Segment에 연결하였다. en=0일 때는 display만 꺼지고 count_out 값은 유지된다.

5. 고찰

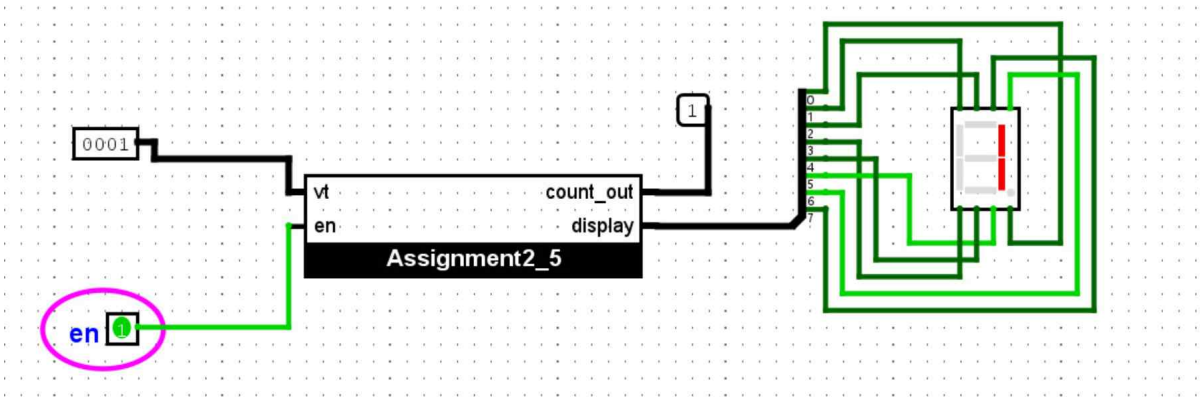
과제 원문은 13비트 vt에서 1의 개수가 10~15일 때 A~F를 표시하도록 요구한다. 제출 코드에는 14→E, 15→F에 해당하는 출력 패턴이 포함되어 있다. 그러나 13비트 입력에서는 1의 개수가 최대 13이므로 E와 F는 입력 vt만으로 발생하지 않는다.

6. 실습 결과

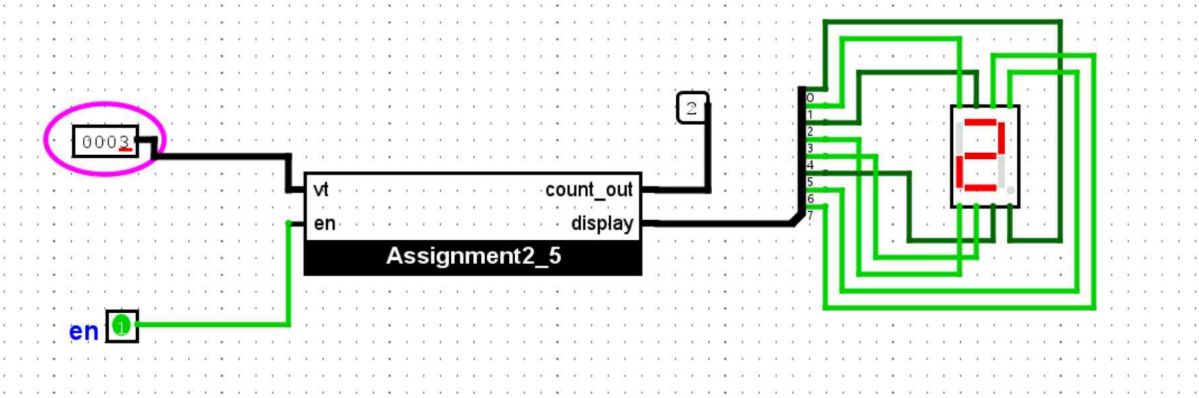
첨부 사진으로 en=1에서 숫자 1~9와 문자 A, b, C, d 표시를 확인하였다. vt=0000 사진은 en=0이므로 숫자 0의 켜진 화면은 포함되어 있지 않다. vt=03FF에서 en=0과 1을 비교하여 표시 소등·활성화 동작도 확인하였다.



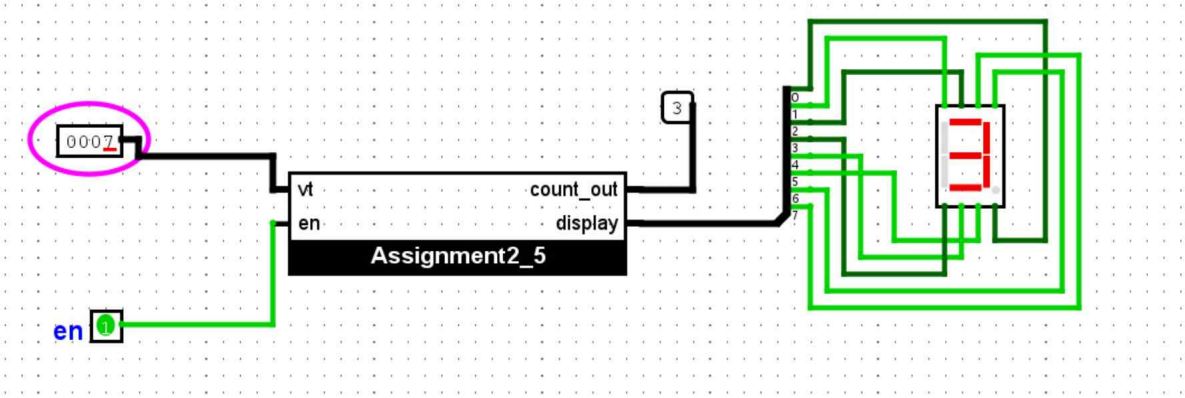
vt = 0000, en = 0 → count = 0, 소등



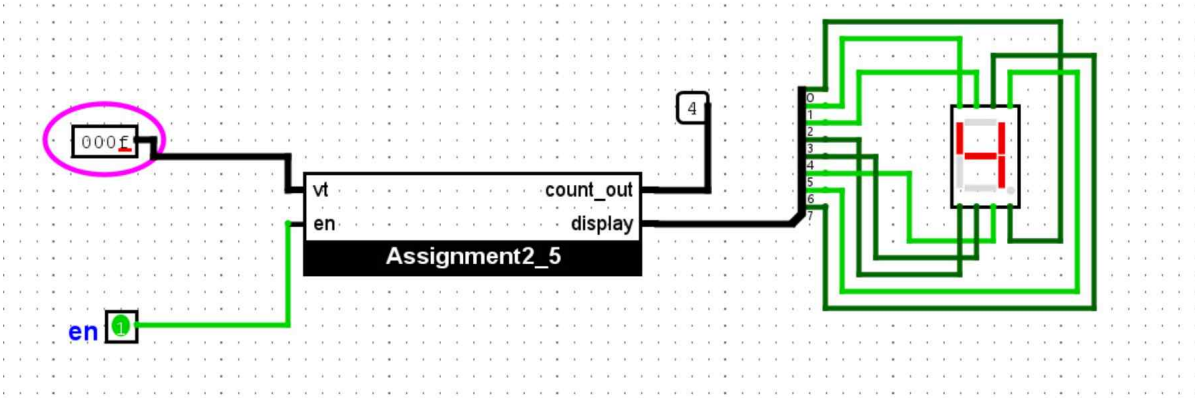
vt = 0001, en = 1 → 1



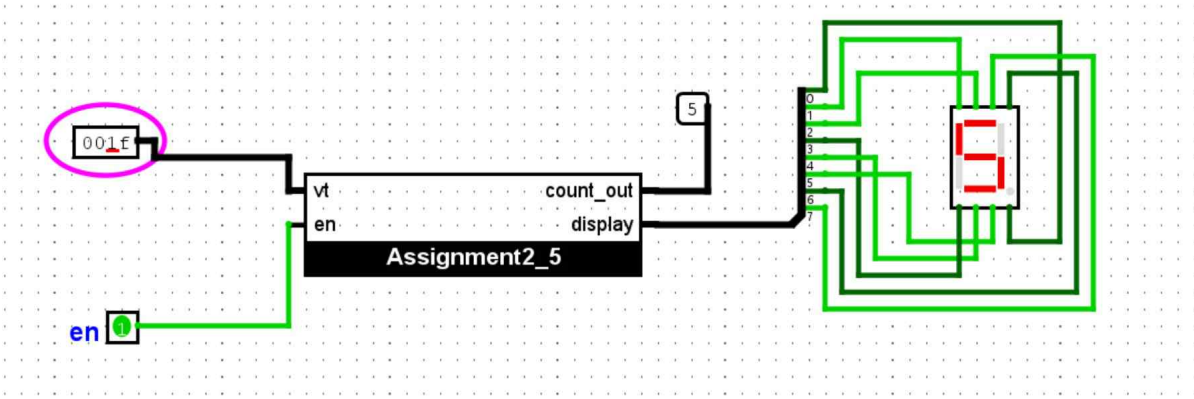
vt = 0003, en = 1 → 2



vt = 0007, en = 1 → 3



vt = 000F, en = 1 → 4



vt = 001F, en = 1 → 5

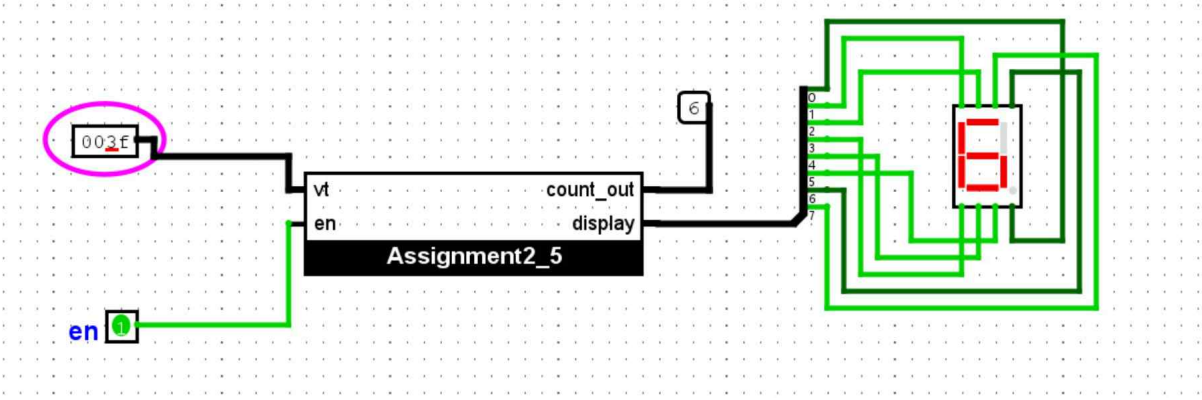


그림 5-7 vt = 003F, en = 1 → 6

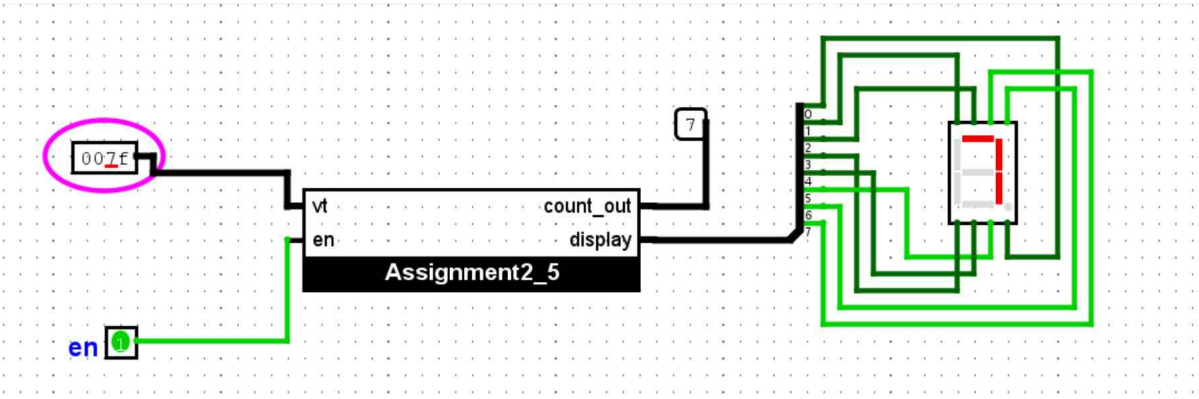


그림 5-8 vt = 007F, en = 1 → 7

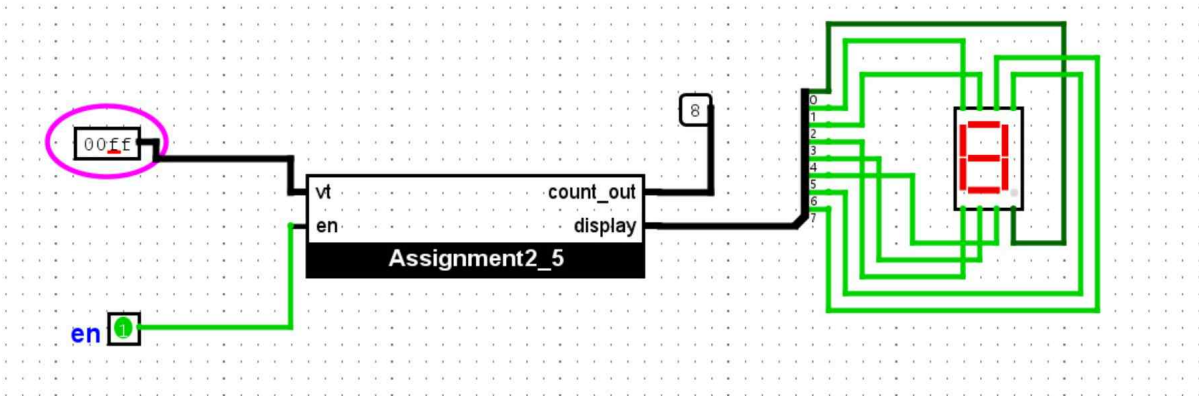


그림 5-9 vt = 00FF, en = 1 → 8

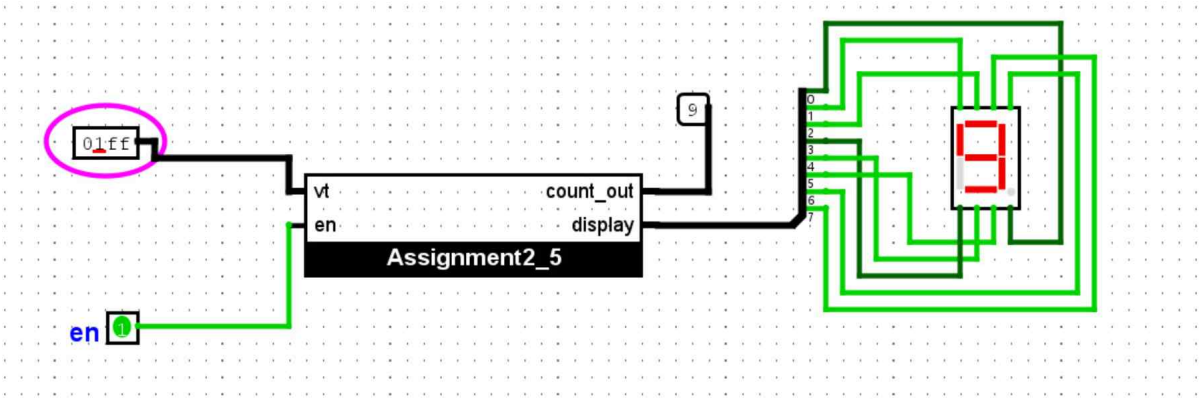


그림 5-10 vt = 01FF, en = 1 → 9

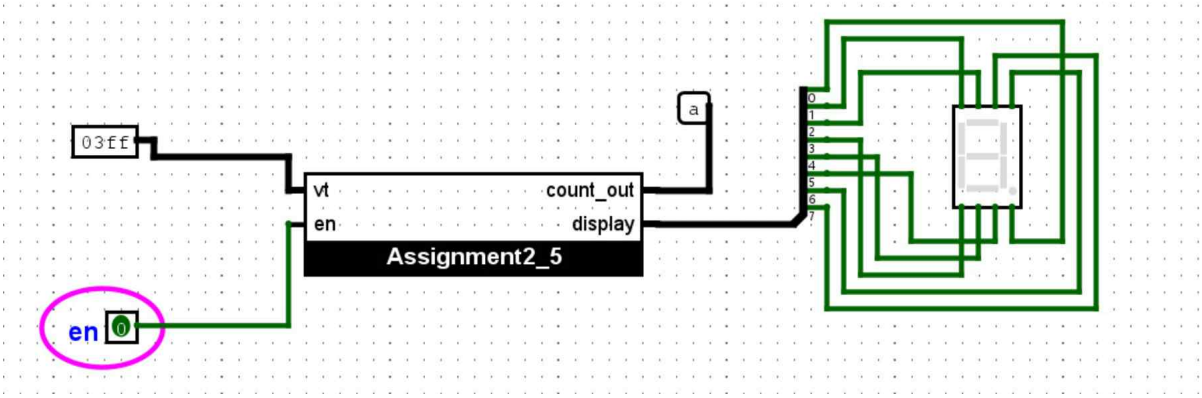


그림 5-11 vt = 03FF, en = 0 → count = A, 소등

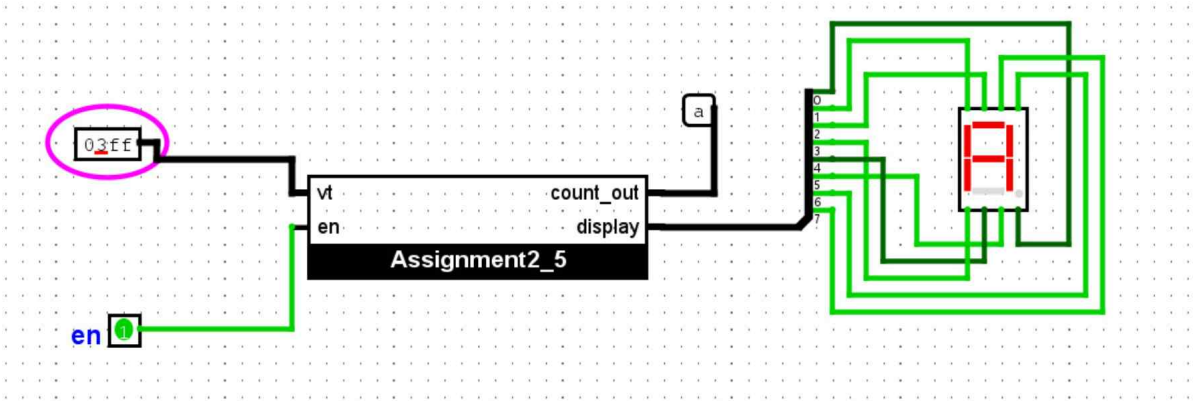


그림 5-A vt = 03FF, en = 1 → A (10개)

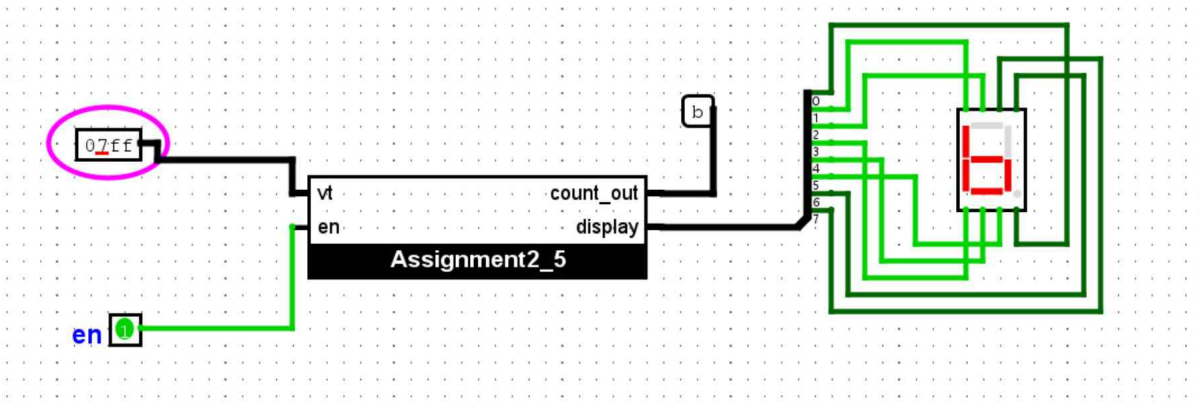


그림 5-B vt = 07FF, en = 1 → b (11개)

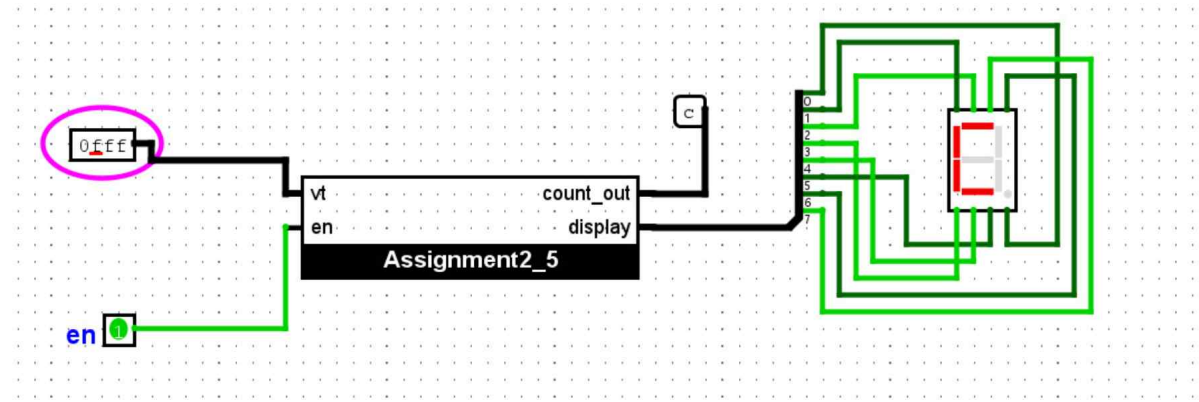


그림 5-C vt = 0FFF, en = 1 → C (12개)

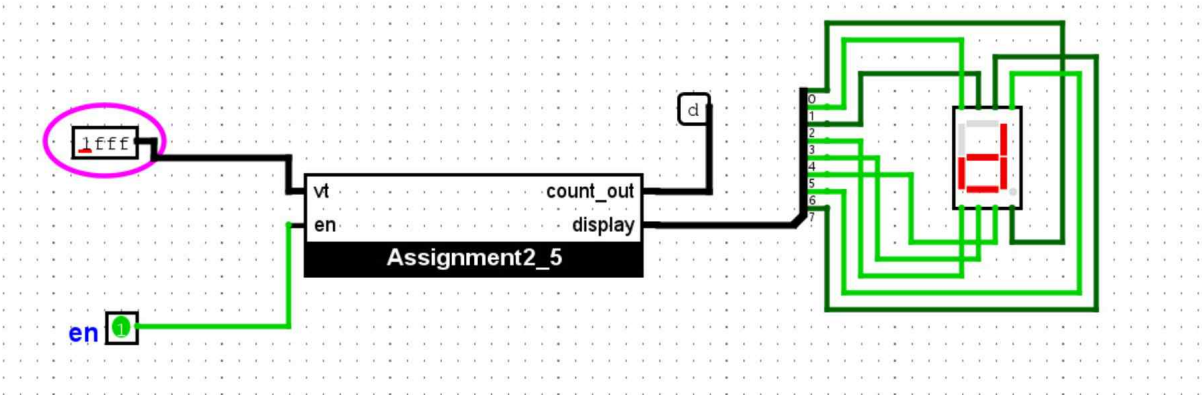


그림 5-D vt = 1FFF, en = 1 → d (13개)